

Airfare Index Project — End-to-End Implementation Plan

Based on the provided project planning document

PART 1 — HOW THE PROJECT STARTS

Before Team A, B or C writes serious code, you need to understand the problem.

Your raw question is:

****What does an Indian traveller actually pay for a flight?****

Suppose today is:

3 September 2026

Someone searches:

Delhi → Mumbai
Travel date → 18 September

Your system might collect:

Source	Airline	Base	Tax	Total
Airline	IndiGo	■4,100	■700	■4,800
Airline	Air India	■4,500	■750	■5,250
OTA	IndiGo	■4,050	■760	■4,810
OTA	Akasa	■3,900	■690	■4,590

Now imagine doing this across:

10-20 routes
x
5 booking windows
x
multiple airlines
x
multiple sources
x
every day

You suddenly have a huge stream of airfare observations.

That is your raw material.

PART 2 — FIRST THING: DECIDE WHAT YOU ARE MEASURING

You cannot just scrape random flights.

You need to define a **representative basket**.

Think about CPI.

CPI doesn't ask:

"What is the price of every single product in India?"

It creates a representative basket.

Your Airfare Index does something similar.

Step 1 — Select representative routes

For example:

DEL → BOM
DEL → BLR
DEL → HYD
DEL → CCU
BOM → BLR
BOM → DEL
BLR → HYD
MAA → DEL
DEL → MAA
BOM → HYD

You should eventually justify these routes using:

- DGCA passenger traffic
- route importance
- flight frequency
- geographic coverage
- official methodology/PSD where applicable

PART 3 — GIVE EACH ROUTE A WEIGHT

Suppose:

Route	Passenger volume
DEL-BOM	10M
DEL-BLR	8M
BOM-BLR	7M
DEL-HYD	5M
Others	20M

You calculate:

$$w_r = \frac{\text{Passenger volume of route}}{\text{Total passenger volume}}$$

So perhaps:

DEL-BOM → 20%
DEL-BLR → 16%
BOM-BLR → 14%
DEL-HYD → 10%
...

Now your index represents something closer to the **Indian domestic aviation market**, rather than an arbitrary average of your favourite routes.

PART 4 — DECIDE THE BOOKING WINDOWS

This is another major part.

For every route, you don't collect only today's fare.

You collect different **advance-purchase windows**.

T+1
T+7
T+15
T+30
T+45

Meaning:

T+1

Flight is tomorrow.

T+7

Flight is seven days from now.

T+15

Flight is 15 days away.

etc.

This lets you study:

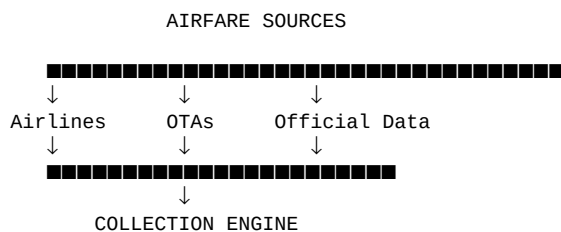
****How does airfare change as the departure date approaches?***

PART 5 — NOW COMES DATA COLLECTION

This is where the actual engineering begins.

Your system connects to permitted sources.

Conceptually:



Potential airline sources:

IndiGo
Air India
Air India Express
Akasa Air
SpiceJet

Potential OTA sources:

MakeMyTrip
Yatra
EaseMyTrip
Cleartrip
Ixigo
Goibibo

But remember:

Your system must respect:

- robots.txt
- Terms of Service
- rate limits

- source policies
- ethical scraping

If a website blocks automated access, **you don't build the project around bypassing its CAPTCHA or anti-bot system.**

Instead:

```

Source unavailable
  ↓
Log failure
  ↓
Use another permitted source
  ↓
Continue pipeline

```

PART 6 — WHAT DOES THE SCRAPER ACTUALLY COLLECT?

Suppose the scraper finds:

```

DEL → BOM
IndiGo
Flight 6E-xxx
18 Sept
T+15
■4,850

```

You don't just save ■4,850.

You save the complete observation.

Something like:

origin	DEL
destination	BOM
airline	IndiGo
flight_number	6E-xxx
search_date	2026-09-03
travel_date	2026-09-18
lead_time	15
fare_class	Economy
base_fare	4100
taxes	700
fees	50
total_fare	4850
source	airline
timestamp	...
availability	available

PART 7 — RAW DATA GOES INTO DATABASE

Don't immediately calculate the index.

First:

```

SCRAPER
  ↓
RAW DATA
  ↓
DATABASE

```

Your database might contain:

```

1,000
10,000
100,000+

```

fare observations.

At this point, the data is **not trustworthy yet**.

PART 8 — NOW CLEAN THE DATA

This is a major stage.

Imagine your scraper gives:

```
■4,800
■4,850
■4,810
■4,790
■48,000
■4,820
NULL
■0
```

Obviously:

```
■48,000
■0
NULL
```

need investigation.

So the cleaning pipeline does:

```
Raw Data
↓
Schema Validation
↓
Duplicate Removal
↓
Missing Value Handling
↓
Invalid Fare Removal
↓
Sold-out Filtering
↓
Cancellation Handling
↓
Outlier Detection
↓
Fare Normalization
↓
Clean Data
```

PART 9 — BASE FARE VS TOTAL FARE

This is especially important.

Suppose:

Base fare	■4,000
Taxes	■700
UDF	■100
Convenience fee	■50

Total	■4,850

Your database should preserve all components.

Why?

Because later you may want to answer:

Did the airfare itself increase?

or:

Did taxes/fees cause the increase?

So don't store only:

```
price = 4850
```

Store:

```
base_fare
taxes
fees
total_fare
```

PART 10 — NOW CREATE A REPRESENTATIVE PRICE

You might have 20 observations for:

```
DEL → BOM
T+7
```

For example:

```
4800
4900
4750
5100
5000
...
```

You need one representative value.

Depending on your statistical methodology, you might use:

- median
- trimmed mean
- weighted mean
- geometric mean

For your prototype, a robust approach can be:

```
P_{r,t} = Median(\text{valid fare observations})
```

So:

```
DEL-BOM
T+7
↓
■4,920
```

PART 11 — NOW CALCULATE THE ROUTE INDEX

This is where your project becomes an **index project**, rather than merely a flight-price project.

Suppose base-period price is:

```
DEL-BOM
Base = ■4,000
```

Today:

```
■5,000
```

Then:

Index = 125

DEL-BOM airfare is approximately ****25% above the base-period level****.

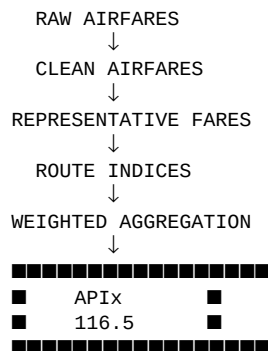
Suppose:

RouteIndexWeight		
DEL-BOM	125	0.30
DEL-BLR	115	0.25
BOM-BLR	110	0.20
DEL-HYD	120	0.15
Others	105	0.10

$$API_x = \sum w_r I_r$$
$$\begin{aligned} \text{API}_X = & 0.30(125) + \\ & 0.25(115) + \\ & 0.20(110) + \\ & 0.15(120) + \\ & 0.10(105) \end{aligned}$$

For example:

At this point you have:



Page 7

Everything else is built around this.

PART 14 — NOW AI ENTERS

This is where your AI team comes in.

AI should **not** calculate the fundamental index.

The statistical engine should.

AI provides intelligence **around the index**.

AI PART 1 — ANOMALY DETECTION

Suppose:

Normal DEL-BOM fare:

■ 4,800
■ 4,900
■ 5,000
■ 4,850
■ 4,950

Suddenly:

■ 12,800

AI detects:

■ Unusual fare spike.

Possible model:

Isolation Forest

Output:

Route: DEL-BOM
Observed: ■ 12,800
Expected: ■ 5,000
Deviation: +156%
Anomaly Score: 0.94

AI PART 2 — FORECASTING

Now you have 30+ days of observations.

You can ask:

What could happen to airfare next week?

Start with:

Moving Average
Exponential Smoothing
ARIMA

Then optionally:

XGBoost

Your output:

Current APIx = 116.5

Forecast:

Tomorrow	117.2
+3 days	118.0
+7 days	119.4

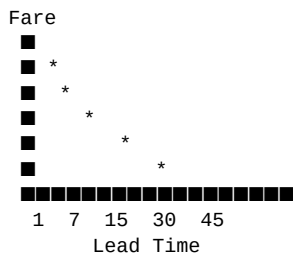
AI PART 3 — LEAD-TIME ANALYSIS

This is one of the coolest parts of your project.

Suppose:

T+45	→	■	4,500
T+30	→	■	4,800
T+15	→	■	5,300
T+7	→	■	6,200
T+1	→	■	8,900

Your system can visualize:



Now you're showing:

****How airfare responds to booking lead time.****

That's much more valuable than simply predicting a number.

PART 15 — NOW BACKTEST EVERYTHING

This is where you prove the system isn't just a pretty dashboard.

You take historical data.

Pretend:

"We are living 30 days ago."

Calculate your index.

Then compare it with actual/reference data.

Your APIx
vs
Benchmark

Evaluate:

Index

- correlation
- MAE
- RMSE
- directional accuracy

Forecast

- MAE
- RMSE
- MAPE

Anomaly detection

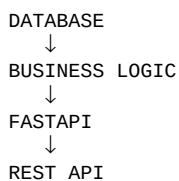
- precision
- recall
- F1

...

PART 16 — NOW THE BACKEND

Once the data + statistical + AI engines work, the backend exposes them.

Architecture:



For example:

```
GET /api/index/current
```

returns:

```
{
  "index": 116.5,
  "daily_change": 1.8,
  "monthly_change": 4.2
}
```

Another:

```
GET /api/routes/DEL-BOM
```

returns route information.

Another:

```
GET /api/forecast/DEL-BOM
```

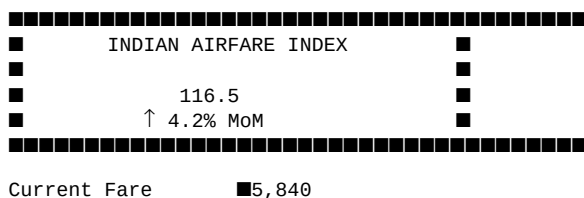
returns forecasts.

...

PART 17 — THEN THE FRONTEND

Now the frontend consumes your APIs.

The dashboard shows:



Routes	15
Quotes	12,842
Sources	4

30 DAY APIx TREND

■ GRAPH

ROUTE HEATMAP

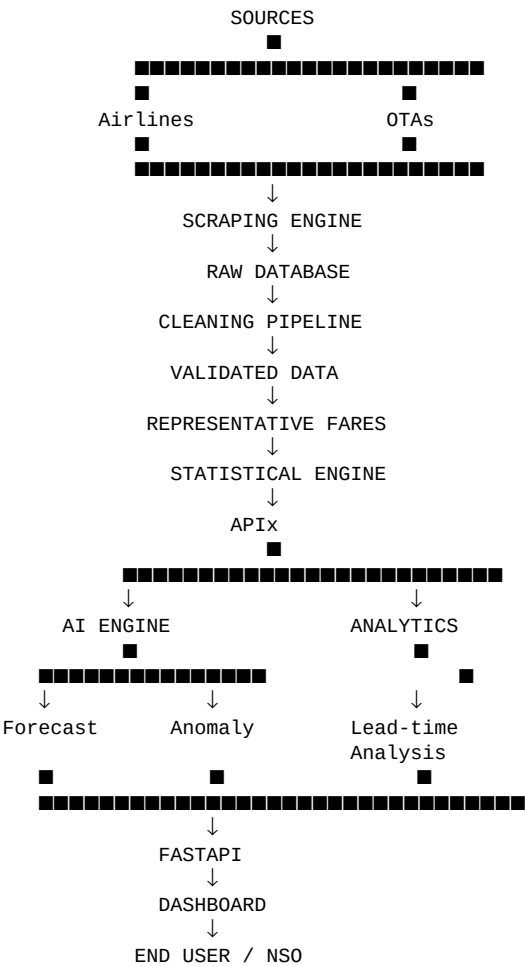
DEL-BOM	+18%
DEL-BLR	+12%
BOM-BLR	-4%

AI INSIGHTS

- DEL-BOM fare spike
- APIx expected to rise
- Weekend prices elevated

PART 18 — FINALLY YOU HAVE A COMPLETE PRODUCT

Your final architecture becomes:



NOW — HOW YOUR 3 TEAMS FIT INTO THIS

This is the part you specifically wanted.

Instead of:

"Jatin does this, Akshara does this..."

Think of **three factories**.